Computational Narratology and Algorithmic Authorship: Bridging Narrative Theory with Agentic Architectures in NovelOS

1. Introduction: The Convergence of Narrative Theory and Systemic Intelligence

The domain of creative authorship is currently undergoing a profound structural transformation, migrating from a paradigm of solitary human ideation toward a model of "Centaur" co-creation involving Large Language Models (LLMs). This shift, however, has illuminated a fundamental friction between the organic fluidity of human creativity—often characterized by "discovery writing" or "pantsing"—and the rigid, context-dependent nature of contemporary AI architectures. The "NovelOS" framework, alongside the advanced capabilities of the "Gemini CLI," highlights a critical operational challenge: LLMs suffer from "contextual drift" and "amnesia" when tasked with managing long-form narratives without the imposition of rigid structural constraints.[1] To mitigate these deficiencies, advanced computational frameworks are increasingly integrating established narrative theories, such as the Story Grid and Dramatica, directly into agentic workflows.[3]

This report provides an exhaustive investigation into the convergence of these disparate domains. It specifically examines how the "Narrative Context Protocol" (NCP) and the Gemini CLI can operationalize abstract literary theory into executable code and system prompts. By treating narrative structures not merely as pedagogical guidelines but as "Constitutional AI" constraints encoded in AGENTS.md and GEMINI.md files, it becomes possible to bridge the chasm between intuitive creative writing and algorithmic consistency. The following analysis explores six critical research areas: the enablement of "safe" discovery writing via branching time architectures, the encoding of the "Five Commandments" into system prompts, the management of epistemic divergence in mystery plots, the automation of intimacy protocols in romance, the mitigation of contextual rot through "Kohärenz Digests," and the optimization of Retrieval Augmented Generation (RAG) for scientific grounding in Climate Fiction (Cli-Fi).

1.1 The Crisis of Context in Generative Narrative

The central problem facing generative narrative is the "Context Window" limitation, which serves as a hard boundary on the cognitive persistence of the Systemic Author. While models like Gemini 1.5 Pro boast million-token windows, the *effective* retention of narrative nuance degrades as the context fills with "noise"—discarded drafts, conversational detritus, and non-canonical exchanges.[5] This degradation manifests as "Contextual Rot," where the model

begins to hallucinate contradictions to established facts (the "Hard Canon") or loses the distinctive voice of the characters.[1]

Traditional writing software, such as Scrivener or Ulysses, addresses the complexity of the novel through static organization—binders, folders, and corkboards.[7] These tools are passive repositories of information. In contrast, the "NovelOS" framework posits an *active* narrative operating system where the structural constraints are enforced by the environment itself. The "Narrative Context Protocol" (NCP) acts as the schema for this environment, explicitly separating the "Subtext" (the underlying structural logic) from the "Storytelling" (the surface-level prose).[2] This separation allows the "Systemic Author" to manipulate the deep structure of the story without rewriting the prose, or conversely, to regenerate the prose while guaranteeing adherence to the structural intent.

1.2 The "Centaur" Model and Algorithmic Constraints

The "Systemic Author" is not a replacement for the human writer but an augmentation of the editorial function. In the "Centaur" model, the human provides the "Creative Impulse" (the *Idea*), while the AI provides the "Generative Horsepower" (the *Draft*). However, without a "bit" and "bridle"—the structural constraints provided by frameworks like Story Grid or Dramatica—the AI's generation is prone to wandering. The integration of "Constitutional AI" principles via the AGENTS.md file creates a bounded creative space.[8] Within these bounds, the AI is free to innovate, but it cannot violate the "Tribal Knowledge" of the story world. This report details how to construct these boundaries using the specific technical affordances of the Gemini CLI and the theoretical affordances of modern narratology.

2. The "Systemic Author" vs. The "Pantser": Algorithmic Architectures for Discovery Writing

Traditional creative writing pedagogy has long bifurcated authors into two distinct psychological profiles: "Plotters" (Architects) and "Pantsers" (Discovery Writers). Plotters rely on rigid outlines, extensive pre-planning, and structural integrity before writing a single word of prose. Pantsers, conversely, write organically, allowing the story to emerge from the characters' immediate actions and the subconscious impulses of the writer. In the context of AI-assisted writing, "pantsing" presents significant risks; without a pre-defined "Storyform," LLMs act as stochastic parrots, generating prose that is locally coherent but globally inconsistent, creating a phenomenon known as "hallucination drift".[1] However, recent advancements in the Gemini CLI, specifically "Shadow Git" and "Checkpointing," offer a technical solution that supports discovery writing within a safety net.

2.1 The Gemini CLI Shadow Git Architecture

The Gemini CLI introduces a "Checkpointing" feature designed to mitigate the risks of destructive edits in software engineering, but this architecture is uniquely suited for narrative discovery. When enabled via settings.json, the CLI maintains a "shadow git repo" located in ~/.gemini/history/<project_hash>.[9] This repository creates a snapshot of the project

state—including the manuscript files and the conversation history—before any tool execution (such as write_file or replace) occurs.[5]

For the "Systemic Author" attempting to "pants," this infrastructure enables a "Branching Time" workflow. In traditional word processors, exploring a "what if" scenario (e.g., "What if the protagonist kills the antagonist in Chapter 3 instead of Chapter 10?") requires manual file duplication or destructive editing. In the Gemini CLI workflow, the author can instruct the AI to generate this narrative fork. If the resulting timeline violates the "Tribal Knowledge" or character constraints, the user can execute the /restore command.[9] This command reverts the file system and the context window to the exact state prior to the divergence, effectively pruning the failed narrative branch.

This capability transforms "pantsing" from a high-risk activity into a low-cost search function. The author can "fuzz test" the plot, generating multiple potential outcomes for a scene to see which one resonates, relying on the Shadow Git to maintain the integrity of the "Hard Canon" (the established history of the story).

2.1.1 Technical Implementation of Branching Time

The mechanics of the Shadow Git are distinct from a standard Git repository used for version control. Standard Git requires explicit commit actions from the user, which disrupts the "flow state" of discovery writing. The Shadow Git operates automatically in the background.

Feature	Standard Git	Gemini Shadow Git	Narrative Implication
Commit Trigger	Manual (git commit)	Automatic (Pre-Tool Execution)	Frictionless experimentation; no need to "save" before a risky narrative choice.
Storage Location	Project Root (.git)	User Home (~/.gemini/history)	Keeps the manuscript clean of experimental debris; separates "Drafting" from "Versioning."
Scope	Files only	Files + Chat History + Tool Calls	Reverting a scene also reverts the *memory* of the AI, preventing "hallucination hang-overs."
Restoration	git checkout	/restore command	Simple, natural language interface for narrative backtracking.

The preservation of Chat History is the critical differentiator for AI-assisted writing. If an author uses standard Git to revert a file, the LLM's context window still contains the conversation about the *deleted* text. This leads to the AI referencing events that no longer

exist in the manuscript. By using /restore, the Gemini CLI ensures that the AI's internal state is synchronized with the document state.[5]

2.2 AGENTS.md as "Tribal Knowledge" and Constitutional Constraint

While the Shadow Git provides the mechanism for temporal branching, the AGENTS.md file provides the semantic constraints that prevent the AI from breaking character during these explorations. The AGENTS.md file serves as a "dedicated, predictable place to provide the context and instructions" for the AI agent.[8] In the "NovelOS" framework, this file functions as the repository for "Tribal Knowledge"—the immutable facts of the story world that persist regardless of the current narrative branch.

By encoding character voices, world rules, and thematic constraints into AGENTS.md, the author establishes a "Constitutional AI" framework. Even when "pantsing," the AI is bound by these directives. For example, if an author attempts a narrative fork where a pacifist character commits murder, the AI, constrained by the "Character Psychology" section of AGENTS.md, may refuse the prompt or flag the inconsistency, acting as a "Self-Reflection Module".[8]

2.2.1 Structuring AGENTS.md for Narrative Consistency

To effectively enforce narrative integrity, the AGENTS.md file must be structured hierarchically. Unlike a standard README, which is for human consumption, AGENTS.md is for machine interpretation.

Proposed AGENTS.md Narrative Schema:

AGENTS.md

1. Constitutional Directives (The Hard Canon)

- **Genre:** Cyberpunk Noir.
- **Tone:** Cynical, atmospheric, high-contrast.
- **Prime Directive:** Agents must never violate the laws of physics defined in physics.md unless explicitly tagged as.

2. Character Profiles (The Cast)

Detective Miller

- **Core Drive:** To find the truth, no matter the cost.
- **Voice:** Laconic, uses short sentences. Avoids contractions when angry.
- **Constraint:** Miller is an alcoholic; he cannot refuse a drink if offered, unless the [Crisis] is immediate.

3. World State (The Simulation)

- **Location:** Neo-Veridia.

- **Current Politics:** The Corporate Council has declared martial law.
- **Magic System:** None. Technology only.

4. Narrative Context Protocol (NCP) Link

- **Storyform:** See storyform.json for current Act/Sequence/Scene structure.
- **Throughlines:**
    - **OS (Overall Story):** The investigation of the murder.
    - **MC (Main Character):** Miller's struggle with addiction.

This structure ensures that even when the user initiates a "pantsing" session (e.g., "Have Miller interview the suspect"), the AI consults the constraints. If the user prompts, "Have Miller fly to the moon," the AI checks Section 3 ("Technology only") and Section 1 ("Laws of Physics") to validate if space travel is established. If not, it flags the deviation.[8]

2.3 Synthesis: The Bounded Discovery Workflow

The integration of Shadow Git and AGENTS.md facilitates a hybrid workflow we term "Bounded Discovery." This workflow resolves the tension between the "Systemic Author" (who requires order) and the "Pantser" (who requires freedom) by creating a "Sandbox" environment.

The workflow proceeds as follows:

1. **Checkpoint:** The Gemini CLI automatically snapshots the state before generation.
2. **Divergence:** The user provides a "What If" prompt to explore a new narrative path.
3. **Constraint Check:** The AI cross-references the prompt against AGENTS.md.
4. **Generation:** If valid, the AI generates the scene.
5. **Review:** The user evaluates the "branch."
    - *Option A (Keep):* The user continues writing. The branch becomes the new Canon.
    - *Option B (Prune):* The user executes /restore. The system reverts to Step 1.

Research Insight: The effectiveness of this workflow relies on the granularity of the checkpoints. The Gemini CLI creates checkpoints *before* file modifications.[9] For narrative purposes, this suggests that the atomic unit of generation should be the "Scene" or "Beat," rather than the "Chapter." By keeping generations granular, the "Systemic Author" maximizes the utility of the /restore command, allowing for rapid iteration of micro-narratives without corrupting the macro-structure. This aligns with the "Agile" methodology in software development, treating the novel as a "codebase" that is iteratively refactored.[6]

3. Encoding "The Five Commandments" into System Prompts

The "Story Grid" methodology posits that every unit of story (Beat, Scene, Sequence, Act) must contain five specific components to be functional: the Inciting Incident, Turning Point, Crisis, Climax, and Resolution.[11] If a scene lacks a Crisis (a choice between "Best Bad" or "Irreconcilable Goods"), it is considered functionally broken; it is merely an event, not a scene. Simultaneously, the "Narrative Context Protocol" (NCP) utilizes a JSON schema to define

"Storypoints" and "Storybeats" for AI agents.[2] The convergence of these frameworks allows for the automation of structural consistency checks via system prompts in GEMINI.md.

3.1 Translating Narrative Theory to JSON Schema

To operationalize the Five Commandments, we must move from qualitative description to quantitative validation. The NCP schema separates "Subtext" (Structure) from "Storytelling" (Presentation).[3] We can define a YAML or JSON schema that represents a "Valid Scene" only if it contains the requisite commandments.

The specific challenge identified in the "Story Grid" is the "Crisis," which requires a clear value shift. A simple prompt asking the AI to "write a scene" often results in aimless dialogue. However, by enforcing a schema that requires the definition of the *Crisis Type* before prose generation, we force the model to reason about the narrative structure.

Proposed GEMINI.md System Prompt Architecture for Story Grid:

Ini, TOML

```
# ~/.gemini/commands/audit_scene.toml
description = "Audits a scene against Story Grid Commandments."
prompt = """
You are a Narrative Architect utilizing the Story Grid methodology.
Analyze the provided scene text or summary.
You must output your analysis in the following strict JSON format.
If the scene fails any commandment, specifically the 'Crisis', reject the draft.

{
  "unit_analysis": {
    "inciting_incident": "String | null",
    "turning_point": {
      "action": "String",
      "value_shift": "String (e.g., +Life to -Death, +Naive to -Worldly)"
    },
    "crisis": {
      "question": "String",
      "type": "Best Bad Choice | Irreconcilable Goods | null",
      "dilemma": "String (Choice A vs Choice B)"
    },
    "climax": "String",
    "resolution": "String",
    "is_functional": "Boolean",
    "missing_elements":
  }
```

}
"""

This prompt utilizes the "Chain of Thought" capabilities of the LLM. By forcing the model to fill out the JSON fields *before* determining is_functional, the system ensures that the evaluation is based on the presence of structural components rather than a vague aesthetic judgment.

3.2 Operationalizing the Self-Reflection Module

The "Self-Reflection Module" is a theoretical component of agentic systems where the model evaluates its own output before presenting it to the user. By integrating the specific language of the Story Grid (e.g., "Value Shift," "Polarity") into the system prompt [11], we ground the LLM's reasoning in a specific ontology.

The "Narrative Triad Architecture" relates to the interaction between the *Agent* (Character), the *Simulation* (Plot), and the *Director* (Authorial Intent). Encoding the Five Commandments into the AGENTS.md or GEMINI.md file ensures that the *Director* agent enforces structural rigor. The research suggests that "reasoning models" (like OpenAI's o3 or Google's Gemini 1.5 Pro) are better suited for this structural analysis than creative generation.[1]

The "Ping-Pong" Workflow:

1. **Generate:** The creative model (e.g., Gemini 1.5 Flash) produces a draft scene based on a prompt.
2. **Audit:** The user runs the /audit_scene command (using a reasoning model like Gemini 1.5 Pro). The command applies the Story Grid schema.
3. **Refine:** If is_functional is false, the reasoning model prompts the creative model to insert the missing commandment (e.g., "The scene lacks a Crisis. Rewrite the dialogue to force the protagonist to choose between their reputation and their safety.").
4. **Finalize:** The corrected scene is presented to the user.

3.3 The Role of Storypoints and Storybeats in NCP

The NCP documentation describes "Storypoints" as the "structured register of narrative features" that encode authorial intent.[2] "Storybeats" are the temporal sequencing of these points. The integration of Story Grid's commandments into NCP involves mapping the "Turning Point" to the NCP's "Story Driver" (Action or Decision).[14]

This mapping is critical because it standardizes the definitions. In Dramatica (the basis for NCP), a "Driver" forces the story from one Act to another.[14] In Story Grid, a "Turning Point" turns the value of a scene. By codifying these definitions in the system prompt, we prevent "Contextual Rot" where the model confuses a "scene climax" with a "global climax."

Table 1: Mapping Dramatica Theory to Story Grid Commandments

Story Grid Commandment	Dramatica Narrative Concept	Implementation in NCP Schema
Inciting Incident	Story Driver (First Driver)	storyform.drivers.incident
Turning Point	Story Driver (Action/Decision)	storybeat.event.driver
Crisis	Dilemma / Justification	storybeat.conflict.dilemma

Climax	Solution / Outcome	storybeat.resolution.outcome
Resolution	New Equilibrium / Judgment	storybeat.state.new_equilibrium

Research Insight: The "Crisis" is the most computationally significant commandment because it represents a binary logic gate (Choice A vs. Choice B). This binary structure makes it highly compatible with "Chain of Thought" reasoning in LLMs. By forcing the AI to explicitly state the binary choice *before* writing the resolution, we significantly increase the logical coherence of the character's actions and ensure the narrative momentum is preserved.

4. Operationalizing "Epistemic Divergence" for Mystery Plots

Mystery narratives rely fundamentally on the asymmetry of information—the difference between what is true (World State) and what the detective or reader believes (Epistemic State). Traditional LLMs operate on a "completion" basis, tending to hallucinate facts or reveal truths prematurely because they do not inherently model "secrets" or "lies." To solve this, we must look to the "Drammar Ontology," which explicitly models "Belief" (BDI model) distinct from "Objective Event".[16]

4.1 The Knowledge Hypergraph in Obsidian

Obsidian, with its graph database capabilities and plugin ecosystem, serves as the ideal substrate for a "Knowledge Hypergraph." The query asks how to structure this to track "Epistemic Divergence." The solution lies in using the "Attribution" class from the BBC Storyline Ontology (or the Drammar equivalent) to tag data.[16]

We can conceptualize the narrative data as a graph where nodes are "Events" or "Facts," and edges are "Attributes."

- **Hard Canon (Objective Truth):** Facts that are universally true in the simulation (e.g., Victim: Killed_By -> Butler).
- **Subjective Storyline (Epistemic Truth):** Facts that are held as true by a specific agent (e.g., Detective: Believes -> Victim: Killed_By -> Gardener).

To operationalize this in Obsidian for an AI agent, we utilize "Frontmatter" (YAML metadata) and "Dataview" or "Canvas" plugins.[11] The "Dataview" plugin allows for dynamic querying of these attributes, enabling the AI to "see" only what the character sees.

Proposed Obsidian Frontmatter Schema for Epistemic States:

YAML

```yaml
---
event_id: EV_045
description: "The poisoning of the tea."
```

```
objective_truth: "The poison was placed in the tea by the Butler at 4:00 PM."
knowledge_access:
  - Agent_Butler: "Knows_Full_Truth"
  - Agent_Killer: "Knows_Full_Truth"
  - Agent_Detective: "False_Belief: Poison was in the wine"
  - Agent_Witness_A: "Partial_Truth: Saw Butler holding a vial"
epistemic_divergence: true
---
```

4.2 The Drammar Ontology and BDI Agents

The Drammar Ontology aligns with the BDI (Belief, Desire, Intention) model.[16] In this framework, a "Belief" is a mental state that may or may not align with the "World State."

- **DramaEntity:** The objective reality.
- **MentalState (Belief):** The subjective reality.

When the Gemini CLI agents access the Obsidian vault (via RAG or context loading), they must be instructed to distinguish between these two layers. If the "Narrative Director" agent asks the "Character Agent" (e.g., the Detective) "Who killed the victim?", the agent must query the knowledge_access field for its own ID. If it finds Agent_Detective: False_Belief, it must generate dialogue consistent with that falsehood.

This requires a sophisticated prompt engineering strategy known as "Perspective Taking." The system prompt must explicitly state: *"You are simulating Agent X. You do not have access to the 'Objective Truth' field. You only have access to 'Agent_X_Knowledge'. Construct your reasoning solely based on that subset."* This effectively "lobotomizes" the omniscient LLM to simulate the limited perspective of a human character.

4.3 Generating Logical Red Herrings

"Red Herrings" are structurally "False Beliefs" that are "Logically Consistent" with the available (but incomplete) evidence. To automate this, the "Knowledge Hypergraph" must support "Inference Chains."

- *Fact:* The Butler has mud on his shoes.
- *Inference (Detective):* Mud implies he was in the garden.
- *Truth:* He stepped in a puddle on the porch.

By modeling these inference chains as edges in the graph, the AI can generate Red Herrings that are not "hallucinations" (random errors) but "calculated deceptions." The AI checks the Objective Truth, identifies the False Belief held by the protagonist, and generates a "Clue" that supports the False Belief.

Research Insight: The management of "Epistemic Divergence" transforms the mystery novel from a linear text into a complex state machine. The "Solution" to the mystery is the convergence of the *Subjective Storyline* with the *Hard Canon*. The narrative arc is the process of pruning the *False Beliefs* until only the *Objective Truth* remains. By explicitly modeling this convergence, the NovelOS ensures that the "Reveal" is both surprising and inevitable.

5. The "Romance Protocol": Automating the Stages of Intimacy

Romance narratives rely on pacing. Rushing from "First Sight" to "Intimacy" breaks the tension and reader investment. The "12 Stages of Intimacy" (Desmond Morris) and "Romancing the Beat" provide a structural roadmap.[20] The challenge is codifying this pacing into a "State Machine" within the "Universal Narrative Model" (UNM) or NCP to prevent the AI from rushing romantic development.

5.1 The Intimacy State Machine

A State Machine is a computational model where the system exists in one "State" at a time, and transitions between states require specific "Inputs" or "Conditions." In the context of the "Relationship Story Throughline" (RS) in Dramatica, the relationship is a character in itself, evolving from one state to another (e.g., "Strangers" to "Partners").[23]

We can codify the "12 Stages of Intimacy" (Eye to Body, Eye to Eye, Voice to Voice, Hand to Hand, etc.) as an enumerated list in the GEMINI.md or AGENTS.md context. This list serves as the "State Transition Diagram" for the romance arc.

Proposed YAML State Machine for Relationship Pacing:

YAML

```yaml
# ~/.gemini/relationship_protocol.yaml
relationship_state:
  current_stage: 3 # Voice to Voice
  stages:
    1: "Eye to Body (Visual Awareness)"
    2: "Eye to Eye (Visual Contact)"
    3: "Voice to Voice (Conversation)"
    4: "Hand to Hand (Casual Touch)"
    5: "Arm to Shoulder (Affectionate Touch)"
    6: "Arm to Waist (Intimate Embrace)"
    7: "Mouth to Mouth (Kissing)"
    8: "Hand to Head (Caress)"
    9: "Hand to Body (Exploration)"
    10: "Mouth to Breast (Advanced Intimacy)"
    11: "Hand to Genitals (Sexual Intimacy)"
    12: "Genitals to Genitals (Intercourse)"
  transition_rules:
    - rule_id: "prevent_rushing"
      precondition: "Must complete 'Voice to Voice' (Scene count > 2) AND 'Hand to Hand'"
```

```
    trigger: "High Emotional Vulnerability Event"
    next_state: 5 # Arm to Shoulder
  - rule_id: "backslide_on_conflict"
    trigger: "Crisis: Betrayal"
    next_state: 2 # Revert to Eye to Eye (Estrangement)
```

5.2 Precondition Checks in Narrative Generation

The "Narrative Director" agent, before generating a scene, must perform a "State Check." If the prompt requests a scene where the characters kiss ("Mouth to Mouth," Stage 7), but the current_stage is 3 ("Voice to Voice"), the "Romance Protocol" acts as a guardrail.
The system prompt in GEMINI.md would include an instruction: *"Consult the relationship_protocol.yaml. If the requested action exceeds current_stage + 1, reject the request as 'Rushing Intimacy' and propose a scene that advances only to the next logical stage."*
This aligns with Dramatica's "Benchmarks".[24] A Benchmark in the Relationship Story Throughline measures the progress of the relationship. By mapping the 12 Stages to the Dramatica Benchmarks (e.g., Act 1 = Stages 1-3, Act 2 = Stages 4-9, Act 3 = Stages 10-12), the AI ensures the romance tracks with the global plot structure.
Research Insight: This "Protocol" prevents the "Insta-Love" hallucination common in LLMs, which are fine-tuned to be "helpful" and "compliant," often resolving tension too quickly. By formally restricting the "Action Space" of the characters based on the "Relationship State," we force the model to generate the "simmering tension" essential to the genre. It effectively forces the AI to "earn" the intimacy through narrative beats rather than jumping to the reward.

6. Managing "Contextual Rot" in Long-Form Series via Kohärenz Digests

As a narrative scales to 50,000+ words, the context window of even the largest LLMs becomes filled with "noise"—early drafts, discarded ideas, and chat logs. This leads to "Contextual Rot," where the model loses track of established facts. Traditional tools use a "Series Bible" (Scrivener Binder).[7] The NovelOS framework proposes "Active Forgetting" and "Context Loading" using the "Kohärenz Digest".[25]

6.1 The Kohärenz Digest Workflow

"Kohärenz" (Coherence) refers to the logical connection of ideas. In the legal context snippet [26], "Kohärenz" implies consistency across different legal frameworks. In narrative, it implies consistency across narrative time. A "Kohärenz Digest" is a compressed summary of the "Narrative Context" that retains causal dependencies while discarding "Surface Storytelling".[25]
The "contextual rot" phenomenon occurs because raw text is "lossy" when tokenized for long-term memory. A "Digest" replaces the raw prose of Chapter 1 with a "Semantic Graph" or "Summary Bullet Points" when generating Chapter 10. This is effectively a compression

algorithm for narrative meaning.

The "Garbage Collection" Mechanism:

1. **Drafting Phase:** The user and AI generate 5,000 words of chat/prose in a session.
2. **Commit Phase:** Upon completion of a scene, the AI runs a summary routine (custom slash command /digest). This command distills the prose into atomic facts.
3. **Update Phase:** The output of /digest (Atomic Facts: "Hero lost sword," "Villain revealed scar") is written to the "Hard Canon" file in the Obsidian Vault (Canon.md or structured JSON).
4. **Flush Phase:** The chat session is cleared/archived. The context window is reset.
5. **Reload Phase:** The next session begins by loading *only* the Canon.md and the AGENTS.md.

This workflow mimics the "Garbage Collection" in programming languages, freeing up memory (tokens) by removing objects (text) that are no longer reachable or necessary for the current operation, while keeping the "Global State" (Canon) intact.

6.2 Files-to-Prompt and RAG

The query asks about the efficacy of using "files-to-prompt" tools versus manually updating a Scrivener Bible. The automated workflow is superior for AI agents because it reduces "Human-in-the-Loop" latency and error.

Using the Gemini CLI's context files (which can be dynamically updated via scripts), the "Kohärenz Digest" ensures that the GEMINI.md always contains the *current* state of the world.[8] This is effectively a "Manual RAG" (Retrieval Augmented Generation). Instead of relying on a vector database to fuzzy-match relevant history (which often fails with subtle narrative details), the "Digest" explicitly curates the "Working Memory."

Research Insight: The "Kohärenz Digest" replaces the "Series Bible" by making the Bible *executable*. In Scrivener, the Bible is a static reference for the *author*. In NovelOS, the Digest is the *source code* for the *agent*. This distinction is vital for autonomous agents; they cannot "flip pages" to check a fact. The fact must be resident in their active memory (context window) to be actionable.

7. Cli-Fi and the "Reality Check" Tool: Optimizing RAG for Scientific Grounding

Climate Fiction (Cli-Fi) demands a high degree of scientific verisimilitude. The "Reality Check" tool utilizes the Gemini CLI's built-in google_web_search and web_fetch capabilities to perform "Retrieval Augmented Generation" (RAG) that validates narrative events against physical reality.[28]

7.1 The /science_audit Custom Command

The query suggests a custom slash command /science_audit. This can be implemented as a .toml or TypeScript extension in Gemini CLI that chains specific tools.[31] The command acts as a "Fact-Checking Agent" that runs in parallel to the "Creative Agent."

Workflow for /science_audit:

1. **Input:** The user highlights a generated scene (e.g., "The protagonist survives a wet-bulb temperature of 38°C for 6 hours using a standard evaporative cooler").
2. **Parsing:** The agent identifies the key physical claims (Temp: 38°C Wet-Bulb, Duration: 6h, Tech: Evaporative Cooler).
3. **Verification (Tool Use):**
    - google_web_search(query="survivability limit wet bulb temperature human")
    - google_web_search(query="do evaporative coolers work in high humidity wet bulb conditions")
4. **Analysis:** The agent compares the search results (Scientific Fact: 35°C wet-bulb is the theoretical limit; evaporative coolers fail at high wet-bulb temps because evaporation ceases) against the Narrative Claim.
5. **Output:** "Reality Check Failed: 38°C Wet-Bulb is fatal. Evaporative cooling is physically impossible in these conditions. Suggested Revision: Use a desiccant-based cooling system or lower the temperature."

7.2 Balancing Accuracy with Narrative Pacing

The challenge in Cli-Fi is not just accuracy, but integration. A dry recitation of facts ("Info-Dumping") kills pacing.[32] The "Reality Check" tool must therefore be paired with a "Narrative Synthesis" prompt.

After the /science_audit returns a correction, the "Narrative Director" agent must be prompted to *dramatize* the science. Instead of a narrator explaining wet-bulb temperatures, the agent rewrites the scene to show the *failure* of the evaporative cooler, increasing the tension (Crisis).

Research Insight: This application of RAG moves beyond "Information Retrieval" to "Logic Validation." We are not just asking the AI to "know" facts, but to "audit" its own imagination. This turns the "Hallucination" problem into a feature: the AI imagines a scenario, the Tool checks if it's possible, and the discrepancy drives the conflict of the story. It ensures that the dystopian elements of the Cli-Fi narrative remain grounded in the terrifying reality of climate science, enhancing the horror and urgency of the genre.

8. Conclusion: The Hybrid Architecture of the Future Novelist

The research questions presented in this report point toward a unified theory of "Algorithmic Authorship" that transcends the binary of "Man vs. Machine." The "Systemic Author" does not surrender creativity to the AI; rather, they construct the *architecture* (NovelOS) within which the AI operates.

- **Shadow Git** allows for fearless exploration ("Pantsing") within a branching time multiverse.
- **The Five Commandments** and **NCP Schemas** provide the syntax and grammar that turn raw text into structured story.

- **Epistemic Graphs** allow for complex deception and mystery by modeling "Theory of Mind."
- **Intimacy Protocols** enforce emotional pacing through state-based logic.
- **Kohärenz Digests** solve the context window limit by converting narrative into data.
- **Reality Checks** ground the fiction in physical plausibility.

By leveraging the "Gemini CLI" and "Narrative Context Protocol," authors can move beyond the "Contextual Drift" of standard LLM interactions and build "Constitutional Narrative Systems"—AI partners that are creative, consistent, and architecturally sound. This represents the maturity of Generative AI from a novelty toy into a professional instrument for complex, long-form storytelling.

Methodology	Technical Implementation	Narrative Benefit
Discovery Writing	Shadow Git / Checkpointing (/restore)	Safe experimentation ("Fuzz Testing" the plot).
Structure	JSON Schemas / System Prompts	Automated pacing and structural integrity.
Mystery	Knowledge Hypergraph (Obsidian)	Consistent Red Herrings and hidden truths.
Romance	State Machine (YAML)	Emotional resonance and pacing control.
Series Memory	Kohärenz Digest / Garbage Collection	Prevents amnesia in long-form works.
Realism	RAG / Google Search Tools	Scientific accuracy and grounded world-building.

This convergence defines the "NovelOS": an operating system for story where the human provides the "Source Code" (Intent) and the AI compiles the "Binary" (Prose).

Referenzen

1. Dramatica Narrative Platform Docs | Dramatica, Zugriff am Januar 2, 2026, https://platform.dramatica.com/
2. Narrative Context Protocol: an Author-centric Storytelling Framework for Generative AI, Zugriff am Januar 2, 2026, https://arxiv.org/html/2503.04844v4
3. narrative-first/narrative-context-protocol: A standardized ... - GitHub, Zugriff am Januar 2, 2026, https://github.com/narrative-first/narrative-context-protocol
4. Introducing the Narrative Context Protocol: Preserving Storytelling ..., Zugriff am Januar 2, 2026, https://narrativefirst.com/blog/introducing-the-narrative-context-protocol-preserving-storytelling-across-ai-agents
5. Who needs Git when you have 1M context windows? - Hacker News, Zugriff am Januar 2, 2026, https://news.ycombinator.com/item?id=45462877
6. Agent Factory Recap: Deep Dive into Gemini CLI with Taylor Mullen | Google Cloud Blog, Zugriff am Januar 2, 2026, https://cloud.google.com/blog/topics/developers-practitioners/agent-factory-rec

ap-deep-dive-into-gemini-cli-with-taylor-mullen
7. Mac- How to Start Writing a Novel Scrivener Template & Outline | Plot Planner - Etsy, Zugriff am Januar 2, 2026, https://www.etsy.com/listing/1166387984/mac-how-to-start-writing-a-novel
8. AGENTS.md, Zugriff am Januar 2, 2026, https://agents.md/
9. Checkpointing - Gemini CLI, Zugriff am Januar 2, 2026, https://geminicli.com/docs/cli/checkpointing/
10. Conductor: Introducing context-driven development for Gemini CLI, Zugriff am Januar 2, 2026, https://developers.googleblog.com/conductor-introducing-context-driven-development-for-gemini-cli/
11. Workflows with Markdown - Mac & Windows (Desktop) - Aeon Timeline, Zugriff am Januar 2, 2026, https://forum.timeline.app/t/workflows-with-markdown/40?page=2
12. What's Your "Can't Live Without It" Tool for Character Development? : r/writers - Reddit, Zugriff am Januar 2, 2026, https://www.reddit.com/r/writers/comments/1oc3vpo/whats_your_cant_live_without_it_tool_for/
13. Narrative Context Protocol: An Open-Source Storytelling Framework for Generative AI - arXiv, Zugriff am Januar 2, 2026, https://arxiv.org/pdf/2503.04844
14. The Fault In Our Stars: An Anatomy Of An Analysis - Articles - Narrative First, Zugriff am Januar 2, 2026, https://narrativefirst.com/articles/the-fault-in-our-stars-an-anatomy-of-an-analysis
15. "Song of the Sea" analysis - Dramatica Platform Community, Zugriff am Januar 2, 2026, https://discuss.dramatica.com/t/song-of-the-sea-analysis/340
16. The GOLEM Ontology for Narrative and Fiction - MDPI, Zugriff am Januar 2, 2026, https://www.mdpi.com/2076-0787/14/10/193
17. Drammar: a comprehensive ontology of drama - Cirma - UniTo, Zugriff am Januar 2, 2026, https://www.cirma.unito.it/drammar/drammarlode/
18. What do you use for your writing? : r/writers - Reddit, Zugriff am Januar 2, 2026, https://www.reddit.com/r/writers/comments/188avcl/what_do_you_use_for_your_writing/
19. I want to use a grid-like system to plan a novel, can I do this in Obsidian? - Reddit, Zugriff am Januar 2, 2026, https://www.reddit.com/r/ObsidianMD/comments/1hn8aoo/i_want_to_use_a_gridlike_system_to_plan_a_novel/
20. Archetypes MAP | PDF | Reason | Storytelling - Scribd, Zugriff am Januar 2, 2026, https://www.scribd.com/document/416968661/Archetypes-MAP
21. The Real Money Is in Customer Value Optimization with Ryan Deiss - Marketing Speak®, Zugriff am Januar 2, 2026, https://www.marketingspeak.com/the-real-money-is-in-customer-value-optimization-with-ryan-deiss/
22. T&C Official Notes | PDF | Sales | Marketing - Scribd, Zugriff am Januar 2, 2026, https://www.scribd.com/document/502467408/T-C-Official-Notes

23. James Hull - Write Brothers, Inc., Zugriff am Januar 2, 2026, https://www.write-bros.com/james-hull.html
24. Building a Story Outline for NaNoWriMo - Series of Articles - Narrative First, Zugriff am Januar 2, 2026, https://narrativefirst.com/articles/series/building-a-story-outline-for-nanowrimo/
25. Graph Representation of Narrative Context: Coherence Dependency via Retrospective Questions - arXiv, Zugriff am Januar 2, 2026, https://arxiv.org/html/2402.13551v1
26. Upcoming issues of EU law - Workshop 24 September 2014 - European Parliament, Zugriff am Januar 2, 2026, https://www.europarl.europa.eu/document/activities/cont/201409/20140924ATT89662/20140924ATT89662EN.pdf
27. Social and Scientific Uncertainties in Environmental Law - Bournemouth University Research Online [BURO], Zugriff am Januar 2, 2026, https://eprints.bournemouth.ac.uk/39745/1/Social%20and%20Scientific%20Uncertainties%20in%20Environmental%20LawFIN.pdf
28. Hands-on with Gemini CLI - Google Codelabs, Zugriff am Januar 2, 2026, https://codelabs.developers.google.com/gemini-cli-hands-on
29. Practical Gemini CLI: Tool calling | by Prashanth Subrahmanyam | Google Cloud - Medium, Zugriff am Januar 2, 2026, https://medium.com/google-cloud/practical-gemini-cli-tool-calling-52257edb3f8f
30. Web search tool (`google_web_search`) | Gemini - Gemini CLI, Zugriff am Januar 2, 2026, https://geminicli.com/docs/tools/web-search/
31. Gemini CLI: Custom slash commands | Google Cloud Blog, Zugriff am Januar 2, 2026, https://cloud.google.com/blog/topics/developers-practitioners/gemini-cli-custom-slash-commands
32. Writing Vivid Dialogue Professional Techniques For Fiction Authors (Hall, Rayne Syverson Etc. | PDF | Adverb | Body Language - Scribd, Zugriff am Januar 2, 2026, https://www.scribd.com/document/782098937/Writing-Vivid-Dialogue-Professional-Techniques-for-Fiction-Authors-Hall-Rayne-Syverson-Etc